

./media/logo-upt.png

Figure 1: ./media/logo-upt.png

UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingeniería de Sistemas

Proyecto de Antivirus

Curso: *Calidad y Pruebas de Software*

Docente: *Mag. Patrick Cuadros Quiroga*

Integrantes:

LLica Mamani, Jimmy Mijair (2023076789)

Sierra Ruiz, Iker Alberto (2023077090)

Tacna – Perú

2026

Sistema *SecureGuard Antivirus*

Informe de Arquitectura de Software

Versión *1.0*

CONTROL DE
VERSIONES

Versión	Hecha por	Revisada por	Aprobada por	Fecha
1.0	LLica Mamani, Jimmy Mijair	Sierra Ruiz, Iker Alberto	LLica Mamani, Jimmy Mijair	01/05/2026

ÍNDICE GENERAL

1. Introducción
2. Descripción de la Arquitectura
3. Diagrama de Clases
4. Diagrama de Base de Datos
5. Diagrama de Componentes

6. Diagrama de Despliegue
7. Infraestructura del Sistema
8. Decisiones de Diseño Arquitectónico
9. Conclusiones

Informe de Arquitectura de Software

1. Introducción

1.1. Propósito

El presente documento describe la arquitectura de software del sistema SecureGuard Antivirus, especificando la estructura de componentes, los diagramas de clases, la base de datos, el despliegue físico y las decisiones de diseño que guiaron la implementación.

1.2. Alcance

La arquitectura documentada cubre el sistema completo: el motor de seguridad (backend Rust), la interfaz gráfica (frontend Python/CustomTkinter) y el mecanismo de comunicación interproceso entre ambos componentes.

1.3. Definiciones Arquitectónicas

Término	Definición
IPC	Inter-Process Communication. Comunicación entre el proceso GUI y el proceso motor mediante stdin/stdout
Daemon mode	Modo de operación del motor donde permanece activo leyendo comandos en bucle
EngineBridge	Capa de abstracción en Python que encapsula toda la comunicación IPC
OFFLINE_HASHES	Variable global estática en Rust que almacena los hashes de firmas cargados al inicio

2. Descripción de la Arquitectura

2.1. Estilo Arquitectónico

SecureGuard Antivirus implementa una arquitectura **cliente-servidor de proceso** con separación en **capas**:

CAPA DE PRESENTACIÓN
 GUI Python (CustomTkinter) - frontend_python/
 Dashboard Protección Firewall Cuarentena ...

CAPA DE INTEGRACIÓN
 EngineBridge - engine_bridge.py
 (JSON sobre stdin/stdout - comunicación IPC)

CAPA DE LÓGICA DE NEGOCIO
 Motor Rust (secureguard_engine) - backend_rust/
 Scanner Quarantine Network Cleaner Updater

CAPA DE DATOS / SISTEMA
 Sistema de Archivos signatures/ Cuarentena (FS)

2.2. Patrón de Comunicación

La comunicación entre la GUI y el motor se realiza mediante **mensajes JSON** a través de las tuberías estándar del proceso hijo:

- **GUI → Motor:** Comandos JSON escritos en `stdin` del proceso motor
- **Motor → GUI:** Respuestas y eventos JSON escritos en `stdout`
- **Formato comando:** {"action": "<comando>", ...parámetros...}
- **Formato respuesta:** {"status": "ok"|"error", "data": {...}}
- **Formato evento:** {"event": "<nombre>", ...datos...}

2.3. Ventajas del Diseño

Característica	Beneficio
Separación de lenguajes	Rust para rendimiento crítico en el motor; Python para agilidad en la GUI
Comunicación asíncrona	La GUI no se bloquea durante operaciones largas de escaneo
Independencia de procesos	El motor puede ejecutarse sin la GUI (modo CLI)
Extensibilidad	Nuevas vistas se agregan sin modificar el motor
Testabilidad	El motor puede probarse de forma independiente con comandos JSON

3. Diagrama de Clases

3.1. Frontend (Python)

```
classDiagram
    class SecureGuardApp {
        +bridge: EngineBridge
        +main_frame: CTKFrame
        +views: dict
        +show_dashboard()
        +show_protection()
        +show_firewall()
        +show_cleaning()
        +show_quarantine()
        +show_update()
        +show_settings()
        +clear_main_frame()
        +on_closing()
    }

    class EngineBridge {
        +engine_path: str
        +process: Popen
        +queue: Queue
        +running: bool
        +callbacks: list
        +lock: Lock
        +start_daemon(signatures_path)
        +stop_daemon()
        +send_command(command, wait, timeout) dict
        +register_callback(callback)
        +unregister_callback(callback)
        +is_running() bool
        +restart_daemon(signatures_path)
        +scan_file(file_path) dict
        +scan_directory(directory_path) dict
        +get_status() dict
        +enable_protection() dict
        +disable_protection() dict
        +get_quarantine_list() dict
        +restore_from_quarantine(file_hash) dict
        +run_cleaner() dict
        +update_signatures() dict
        -_read_output()
        -_monitor_process()
    }
```

```

class DashboardView {
    +bridge: EngineBridge
    +scan_btn: CtkButton
    +progress: CtkProgressBar
    +status_circle: CtkLabel
    +status_text: CtkLabel
    +start_scan()
    +run_scan()
    +scan_finished()
    +on_event(event)
    +show_threat(event)
}

class ProtectionView {
    +bridge: EngineBridge
    +realtime_var: BooleanVar
    +ransom_var: BooleanVar
    +behavior_var: BooleanVar
    +toggle_realtime()
    +toggle_ransomware()
    +toggle_behavior()
}

class FirewallView {
    +bridge: EngineBridge
    +firewall_var: BooleanVar
    +status_label: CtkLabel
    +profile_labels: dict
    +toggle_firewall()
    -_apply_firewall(enabled)
    -_on_result(enabled, result)
    -_update_status_label()
}

class QuarantineView {
    +bridge: EngineBridge
    +listbox: CtkTextbox
    +load_quarantine()
    +restore_selected()
    +delete_selected()
}

class CleaningView {
    +bridge: EngineBridge
    +clean_btn: CtkButton

```

```

        +status_label: CTKLabel
        +start_clean()
        +run_clean()
        +clean_finished(result)
    }

class SettingsView {
    +bridge: EngineBridge
    +__init__(parent, bridge)
}

class UpdateView {
    +bridge: EngineBridge
    +update_btn: CTKButton
    +status_label: CTKLabel
    +start_update()
    +run_update()
    +update_finished(result)
}

SecureGuardApp --> EngineBridge : uses
SecureGuardApp --> DashboardView : creates
SecureGuardApp --> ProtectionView : creates
SecureGuardApp --> FirewallView : creates
SecureGuardApp --> QuarantineView : creates
SecureGuardApp --> CleaningView : creates
SecureGuardApp --> SettingsView : creates
SecureGuardApp --> UpdateView : creates
DashboardView --> EngineBridge : uses
ProtectionView --> EngineBridge : uses
FirewallView --> EngineBridge : uses
QuarantineView --> EngineBridge : uses
CleaningView --> EngineBridge : uses
UpdateView --> EngineBridge : uses

```

3.2. Backend (Rust — módulos principales)

```

classDiagram
    class Threat {
        +file: String
        +hash: String
        +threat_type: String
        +confidence: f32
    }

    class Scanner {

```

```

+OFFLINE_HASHES: Mutex~Vec~String~~
+initialize_with_offline_signatures(sig_dir: str) Result
+scan_directory(path: str) Value
+scan_files(paths: Vec~PathBuf~) Result~Vec~Threat~~
-calculate_entropy(data: Vec~u8~) f64
-is_pe_file(data: Vec~u8~) bool
}

class Daemon {
+EVENT_SENDER: Option~Sender~String~~
+handle_command(cmd_json: str) Value
-start_file_monitor() Value
-process_new_file(path: PathBuf) Result
+send_event(event: Value)
}

class Quarantine {
+QUARANTINE_DIR: str
+quarantine_file(file_path: Path) Result
+restore_file(quar_file: str) Value
+delete_quarantined(quar_file: str) Value
+list_quarantine() Vec~Value~
}

class Network {
+set_firewall(enable: bool) Value
}

class Cleaner {
+clean_temp_files() Value
}

class Updater {
+UPDATE_URL_HASHES: str
+UPDATE_URL_YARA: str
+LOCAL_SIG_DIR: str
+update_signatures() Value
}

class Utils {
+ok_response(data: Value) Value
+error_response(msg: str) Value
}

Daemon --> Scanner : calls
Daemon --> Quarantine : calls

```

```

Daemon --> Network : calls
Daemon --> Cleaner : calls
Daemon --> Updater : calls
Scanner --> Threat : produces
Scanner --> Utils : uses
Quarantine --> Utils : uses
Network --> Utils : uses
Cleaner --> Utils : uses
Updater --> Utils : uses

```

4. Diagrama de Base de Datos

4.1. Modelo de Almacenamiento

SecureGuard Antivirus v1.0 utiliza el **sistema de archivos** como mecanismo de persistencia, sin base de datos relacional. A continuación se describe el modelo de datos del sistema de archivos:

```

erDiagram
    SIGNATURES_DIR {
        string filename "malware_hashes.txt | yara_rules.yar"
        text content "Líneas de hashes SHA-256 o reglas YARA"
        datetime last_updated "Fecha de última actualización"
    }

    MALWARE_HASH {
        string sha256_hash PK "Hash SHA-256 del archivo malicioso (64 chars hex)"
        string comment "Comentario opcional (líneas que empiezan con #)"
    }

    QUARANTINE_ENTRY {
        string quar_filename PK "Nombre: <sha256>.quar"
        binary file_data "Contenido del archivo en cuarentena"
        datetime quarantine_date "Fecha en que fue puesto en cuarentena"
    }

    QUARANTINE_META {
        string meta_filename PK "Nombre: <sha256>.meta"
        string original_path "Ruta original del archivo antes de cuarentena"
    }

    SIGNATURES_DIR ||--o{ MALWARE_HASH : "contiene"
    QUARANTINE_ENTRY ||--|| QUARANTINE_META : "referenciado por"

```

4.2. Estructura de Archivos de Datos

Archivo	Ruta	Formato	Descripción
malware_hashes.txt	signatures/	Texto plano (1 hash/línea)	Firmas SHA-256 de malware conocido
yara_rules.yar	signatures/	YARA syntax	Reglas de detección de familias de malware
<sha256>.quar	C:\ProgramData\BitDefender\Quarantine\	Binario	Archivos maliciosos renombrados y aislados
<sha256>.meta	C:\ProgramData\BitDefender\Quarantine\	Texto plano	Partido original del archivo en cuarentena

4.3. Modelo de Datos Planificado (v2.0)

Para la versión 2.0 se planifica migrar a PostgreSQL con el siguiente esquema:

```
erDiagram
    SIGNATURE {
        uuid id PK
        varchar(64) sha256_hash UK
        varchar(255) threat_name
        varchar(50) threat_family
        timestamp created_at
        timestamp updated_at
        integer version
    }

    SCAN_RESULT {
        uuid id PK
        timestamp scan_date
        varchar(500) scanned_path
        integer total_files
        integer threats_found
        varchar(20) scan_type
    }

    THREAT_DETECTION {
        uuid id PK
        uuid scan_id FK
        uuid signature_id FK
        varchar(500) file_path
        varchar(64) file_hash
        float confidence
        boolean quarantined
    }

    QUARANTINE_RECORD {
```

```

    uuid id PK
    varchar(64) file_hash UK
    varchar(500) original_path
    varchar(255) quarantine_filename
    timestamp quarantined_at
    boolean restored
    timestamp restored_at
}

SIGNATURE ||--o{ THREAT_DETECTION : "identifica"
SCAN_RESULT ||--o{ THREAT_DETECTION : "contiene"
THREAT_DETECTION ||--o| QUARANTINE_RECORD : "genera"

```

5. Diagrama de Componentes

```

graph TB
    subgraph "FRONTEND - Python (frontend_python/)"
        App[" SecureGuardApp\n(main.py)"]
        Bridge[" EngineBridge\n(engine_bridge.py)"]
        subgraph "Views"
            DB[" DashboardView"]
            PR[" ProtectionView"]
            FW[" FirewallView"]
            CL[" CleaningView"]
            QU[" QuarantineView"]
            UP[" UpdateView"]
            SE[" SettingsView"]
        end
    end

    subgraph "COMUNICACIÓN IPC"
        STDIN[" stdin\n(Comandos JSON)"]
        STDOUT[" stdout\n(Respuestas + Eventos JSON)"]
    end

    subgraph "BACKEND - Rust (backend_rust/)"
        Main[" main.rs\n(Dispatcher)"]
        Daemon[" daemon.rs\n(Command Handler)"]
        Scanner[" scanner.rs\n(Motor de Escaneo)"]
        Quarantine[" quarantine.rs\n(Gestión de Cuarentena)"]
        Network[" network.rs\n(Control Firewall)"]
        Cleaner[" cleaner.rs\n(Limpieza de Temp)"]
        Updater[" updater.rs\n(Actualización de Firmas)"]
        Protection[" protection.rs\n(Monitoreo en Tiempo Real)"]
        Utils[" utils.rs\n(Respuestas JSON)"]
    end

```

```

subgraph "ALMACENAMIENTO"
    Sigs[" signatures/\nmalware_hashes.txt\nyara_rules.yar"]
    QuarantineFS[" C:\\ProgramData\\SecureGuard\\Quarantine\n*.quar + *.meta"]
    TempFS[" C:\\Windows\\Temp\nArchivos temporales"]
    WinFirewall[" Windows Firewall\n(netsh advfirewall)"]
end

subgraph "RECURSOS EXTERNOS"
    GitHub[" GitHub\nRepositorio de Firmas"]
end

App --> Bridge
App --> DB
App --> PR
App --> FW
App --> CL
App --> QU
App --> UP
App --> SE

Bridge --> STDIN
STDOUT --> Bridge

STDIN --> Main
Main --> Daemon
Daemon --> Scanner
Daemon --> Quarantine
Daemon --> Network
Daemon --> Cleaner
Daemon --> Updater
Daemon --> STDOUT

Scanner --> Sigs
Quarantine --> QuarantineFS
Cleaner --> TempFS
Network --> WinFirewall
Updater --> GitHub
Updater --> Sigs
Scanner --> Utils
Quarantine --> Utils
Network --> Utils
Cleaner --> Utils
Updater --> Utils

```

6. Diagrama de Despliegue

```
graph TB
    subgraph "Equipo del Usuario (Windows 10/11)"
        subgraph "Proceso: python main.py"
            GUI[" GUI - CustomTkinter\nfrontend_python/main.py\nPuerto: N/A (GUI local)"]
            EB[" EngineBridge\nSubproceso gestionado via\nsubprocess.Popen"]
        end

        subgraph "Proceso hijo: secureguard_engine.exe"
            Engine[" Motor Rust\nsecureguard_engine.exe\nModo: daemon"]
            FileWatcher[" File Watcher\nnotify crate\nMonitorea: Docs/Downloads/Public"]
        end

        subgraph "Sistema de Archivos Local"
            SigsDir[" signatures\nmalware_hashes.txt\nnyara_rules.yar"]
            QuarantineDir[" C:\\ProgramData\\SecureGuard\\Quarantine\\"]
            TempDirs[" C:\\Windows\\Temp\\n%TEMP%"]
        end

        subgraph "Servicios del SO"
            WinFW[" Windows Firewall Service\nnetsh advfirewall"]
            NTFS[" NTFS - Sistema de Archivos"]
        end
    end

    subgraph "Internet"
        GHRepo[" GitHub Repositorio\nSecureGuard-Signatures\n(HTTPS)"]
    end

    GUI -- "stdin/stdout JSON\n(IPC via subprocess)" --> Engine
    Engine --> FileWatcher
    Engine --> SigsDir
    Engine --> QuarantineDir
    Engine --> TempDirs
    Engine --> WinFW
    Engine -- "HTTPS GET" --> GHRepo
    GHRepo -- "malware_hashes.txt\nnyara_rules.yar" --> SigsDir
    FileWatcher --> NTFS
    Engine --> NTFS
```

6.1. Requisitos de Despliegue

Componente	Requisito mínimo	Requisito recomendado
SO	Windows 10 64-bit	Windows 11 64-bit

Componente	Requisito mínimo	Requisito recomendado
CPU	Intel/AMD x64, 2 núcleos	4+ núcleos
RAM	4 GB	8 GB
Disco	500 MB libres	2 GB libres
Python	3.10+	3.12
Internet	No requerido (offline mode)	Recomendado para actualizaciones
Permisos	Administrador (para firewall y cuarentena)	Administrador

7. Infraestructura del Sistema

7.1. Dependencias del Backend (Rust)

Crate	Versión	Función
<code>serde + serde_json</code>	1.0	Serialización/deserialización JSON
<code>walkdir</code>	2.4	Traversal recursivo del sistema de archivos
<code>sha2</code>	0.10	Cálculo de hashes SHA-256
<code>hex</code>	0.4	Codificación hexadecimal de hashes
<code>notify</code>	6.1	Monitoreo de cambios en el sistema de archivos
<code>request</code>	0.11	Cliente HTTP para descarga de firmas
<code>dirs</code>	5.0	Obtención de directorios estándar del SO
<code>chrono</code>	0.4	Manejo de fechas y tiempos
<code>clap</code>	4.4	Parsing de argumentos CLI
<code>zip</code>	0.6	Compresión/descompresión de firmas

7.2. Dependencias del Frontend (Python)

Librería	Versión	Función
<code>customtkinter</code>	5.2.0	Framework de GUI moderno basado en Tkinter
<code>Pillow</code>	10.0.0	Procesamiento de imágenes (logos, íconos)
<code>psutil</code>	5.9.5	Métricas del sistema (CPU, RAM, disco)
<code>plyer</code>	2.1.0	Notificaciones nativas del sistema operativo

Librería	Versión	Función
<code>requests</code>	2.31.0	Cliente HTTP (usado en actualizaciones vía Python)

7.3. Herramientas de Construcción y Empaquetado

Herramienta	Versión	Uso
Rust toolchain	1.75+	Compilación del motor con <code>cargo build --release</code>
Python	3.12	Ejecución del frontend
PyInstaller	6.x	Empaquetado del frontend en ejecutable <code>.exe</code>
Git	2.x	Control de versiones

8. Decisiones de Diseño Arquitectónico

8.1. Decisión 1: Rust para el Motor de Seguridad

Aspecto	Detalle
Decisión	Usar Rust para el motor de escaneo y las operaciones de seguridad
Alternativas consideradas	Python puro, C++, Go
Justificación	Rust ofrece rendimiento nativo sin recolector de basura, garantías de seguridad de memoria en tiempo de compilación, y <i>cero cost abstractions</i> . Crítico para un motor de seguridad que procesa archivos rápidamente
Consecuencias	Mayor complejidad de compilación; requiere toolchain Rust. Beneficio: motor ~10x más rápido que Python equivalente

8.2. Decisión 2: Comunicación IPC vía stdin/stdout JSON

Aspecto	Detalle
Decisión	Protocolo JSON línea-a-línea sobre stdin/stdout del subproceso
Alternativas consideradas	Named pipes, sockets TCP locales, shared memory, gRPC
Justificación	Simplicidad de implementación, portable entre sistemas operativos, no requiere serialización compleja, fácil de depurar (mensajes legibles)
Consecuencias	Limitación de throughput para operaciones masivas; suficiente para v1.0. Para v2.0 se podría migrar a gRPC o sockets Unix

8.3. Decisión 3: CustomTkinter para la GUI

Aspecto	Detalle
Decisión	Usar CustomTkinter como framework de interfaz gráfica
Alternativas consideradas	PyQt6, wxPython, Electron, Tkinter puro
Justificación	CustomTkinter ofrece diseño moderno con tema oscuro nativo, se instala con <code>pip</code> , es compatible con PyInstaller, y no requiere licencias comerciales (PyQt6 requiere GPLv3 o licencia comercial)
Consecuencias	Menor ecosistema de componentes que PyQt; suficiente para los requerimientos de la interfaz actual

8.4. Decisión 4: Sistema de Archivos como Persistencia (v1.0)

Aspecto	Detalle
Decisión	Usar archivos planos (<code>.txt</code> , <code>.quar</code> , <code>.meta</code>) en lugar de base de datos
Alternativas consideradas	SQLite, PostgreSQL, Redis
Justificación	Simplicidad de implementación para MVP, sin dependencias adicionales, funcionamiento offline garantizado

Aspecto	Detalle
Consecuencias	Sin consultas complejas; para v2.0 se planifica migración a PostgreSQL con historial de escaneos y telemetría

9. Conclusiones

1. **Arquitectura en capas:** La separación en capa de presentación (Python), capa de integración (EngineBridge) y capa de lógica de negocio (Rust) proporciona un diseño limpio con responsabilidades bien definidas y bajo acoplamiento entre componentes.
2. **Comunicación IPC robusta:** El protocolo JSON línea-a-línea sobre stdin/stdout es simple, portable y fácil de probar. El EngineBridge gestiona la concurrencia mediante colas y threads separados para lectura, escritura y monitoreo del proceso hijo.
3. **Modularidad del motor:** Cada módulo Rust (`scanner`, `quarantine`, `network`, `cleaner`, `updater`) es independiente y se comunica únicamente a través del dispatcher en `daemon.rs`, facilitando el mantenimiento y las pruebas unitarias.
4. **Escalabilidad planificada:** La arquitectura actual (v1.0) es adecuada para uso en escritorio. El diseño anticipa la migración a infraestructura en nube (AWS/Azure con Terraform, documentada en FD01) y base de datos relacional (PostgreSQL) en la versión v2.0.
5. **Diagramas como contrato:** Los diagramas de clases, componentes y secuencia (en FD03) sirven como contrato entre los desarrolladores del frontend y del backend, definiendo claramente las interfaces de comunicación y los modelos de datos.